Chapter 4. The Cultural Context of SRE

By Ayelet Sachto, Brian Kudzia, Igor Oks, Jennifer Petoff, Luca Cervasio, Nathen Harvey, and Roman Seibel
with Garrett Holthaus

Organizational culture is foundational to the long-term success or failure of an SRE transformation. While SRE relies on technical practices and processes, these are deeply influenced by the environment and people surrounding and implementing them. We aren't saying that having a conducive culture on day one is a prerequisite, but rather that building a supportive culture is helpful in establishing and maintaining a successful SRE practice. This chapter presents data-driven, research-backed arguments for the critical importance of culture. It will help you identify cultural elements that support SRE, articulate why culture is crucial for SRE success, build a plan to establish and foster a culture of reliability, and recognize and mitigate cultural antipatterns.

The Case for Culture

> *The fifth nine is people.*
>
> —Trisha Weir, Google SRE Manager

Let's start by laying out the case for why culture is a powerful catalyst for achieving high reliability for users, happy engineers, and overall organizational success in technology-driven environments.

The "nines" in Weir's quote refer to the number of nines in a reliability percentage: 99.999% availability is referred to as "five nines." This perspective emphasizes that while you can improve reliability through processes and tools, human behavior, collaboration, and the underlying culture are indispensable for reaching and sustaining elite performance. Organizations that achieve this level of maturity can unlock reliability improvements of up to an order of magnitude. Ben Treynor Sloss, the founder of SRE at Google, views SRE teams' ability to be a dedicated voice for reliability as critically impor-

tant, and cultural elements are essential for making SREs effective. Let's explore this further.

Key Elements of a Reliability-Focused Culture

At its core, SRE applies a software engineering mindset to managing services in production. This approach fosters structured, strategic, and reproducible improvements, enabling systems to scale sustainably while reducing reliance on manual effort. The engineering mindset forms the bedrock upon which a robust reliability-focused culture thrives.

What practical steps are needed to cultivate such a culture and turn these principles into day-to-day reality? A high-performing culture focused on reliability is built upon several key elements that prioritize alignment, learning, collaboration, and user satisfaction. These cultural components are crucial for enabling teams to effectively manage complex systems, respond to incidents, and drive continuous improvement. Together, they create the conditions for SRE practices to thrive and deliver meaningful results.

Let's start with the basics. There are two fundamental aspects of a reliability-focused culture:

Focus on the user

SRE is inherently user-centric. It involves a deep understanding of end users' needs and goals, as well as leveraging their signals to continuously improve products and services. The SRE approach is *not* reliability at any cost. Instead, SRE focuses on balancing the expectations of end users and the business along the reliability–velocity spectrum. Maintaining reliable systems is fundamentally about meeting these expectations, which in turn keeps users happy. This requires measuring system health and performance in a way that closely reflects the user experience.

Reliability as a core value

Organizational commitment to reliability is a prerequisite. Reliability is understood not just as a technical goal but as an essential feature of any product or service. It must be valued, recognized, and rewarded across the organization, from engineers to executives, ensuring SRE has a seat at the decision-making table.

These concepts underpin everything else, but they must be supported by specific cultural elements and practices.

Fostering a Reliability-Focused Culture

Seven cultural elements support the fundamental tenets outlined in the previous section, as well as the high standards of excellence for which SRE is known. They are:

- A data-driven mindset
- Collaboration and shared responsibility
- Learning and continuous improvement
- Blamelessness
- Psychological safety
- Trust
- Empowerment

We'll explore each of these in this section, looking across several dimensions that represent the elements of a reliability-focused culture. The two fundamental aspects, "reliability as a core value" and "focus on the user," underpin everything else. Now let's deep dive into *how* the remaining seven cultural aspects foster these two. We'll look across the software development lifecycle at how organizations can integrate reliability-focused culture and practices into their business.

Data-driven mindset

SRE requires a data-driven approach to effectively prioritize efforts and minimize future incidents. You need to establish a variety of metrics—for example, to measure how well you're doing against your targets, alert when there is a problem, and debug problems to find root causes. Adopting a *measurement culture* facilitates this data-driven approach.

To set the right goals, allow for transparency, and build data-driven decision-making processes, you need to measure *everything important or relevant*—this means you need not only reliability metrics but also toil metrics and data that represents the customer experience. This data helps you identify gaps and prioritize solutions based on business needs and customer expectations. For example, the data from postmortems (also known as post-incident reviews, or PIRs) is crucial for identifying patterns and guiding investments in failure analysis.

SRE was created to address the challenges of operating and scaling large production systems at Google. This scaling required not just technical changes but also organizational and human changes. The organization had to be aligned around a shared goal—happy customers—without hindering innovation. This meant aligning incentives across different organizations: business, development, and operations. Achieving this requires a common

language spanning the product lifecycle. In the field of system reliability, that common language is now widely established through concepts such as service level indicators (SLIs) and service level objectives (SLOs). These elements, discussed in Chapter 8, form the bedrock of an SRE measurement culture and contextualize our data-driven mindset.

To innovate rather than be fully occupied with the unachievable goal of "100% reliability," we need a way to numerically estimate the risk ceiling and failure tolerance a system can have and still meet user expectations. We express this as an *error budget*—the amount of downtime our users are willing to tolerate (see Chapter 8). We measure how we are doing against our error budget and use this for data-driven processes, decision making, and work planning. For example, we might decide to freeze rollouts, or to temporarily reassign part of a team to reliability work.

Measurement culture applies not just to reliability metrics but to every aspect of our business that underpins the high standards of excellence to which we hold ourselves. For example, at Google we have a very strong postmortem culture, including detailed reports and tracking of postmortem action items. When we noticed systemic delays in writing the reports and implementing action items, we took a data-driven approach. We collected metrics and set SLOs related to postmortem processes, such as the maximal time to write the first draft and to review and publish it. We also created dashboards, alerts, and management-visible reporting to track adherence to these SLOs. This led to a reduction in report publication time, increased implementation of action items, and a decrease in recurrent "same root cause" incidents.

Similarly, we introduced SLOs (with reporting and clear expectations) for handling high-priority internal issues, including the time to provide internal status updates and the time to resolve the issue. For incidents, we set and enforce goals for metrics such as:

Percentage of unactionable incidents

A high rate of unactionable incidents points to noisy or poorly configured monitoring.

Percentage of manually detected incidents

A high rate of problems first reported by users or other teams suggests incorrectly defined SLIs or inadequate monitoring coverage.

Page per incident ratio

Too many alerts for a single incident might distract an on-caller and make it harder to diagnose what's going on.

We apply the same measurement culture to people operations. To prevent burnout, we track metrics for on-call time. We also track how frequently on-callers have to apply manual changes in production, and we strive to keep this number as close to zero as possible.

Ultimately, a data-driven mindset is the basis for everything SREs do, from measuring performance to driving effective prioritization and collaboration.

Collaboration and shared responsibility

The broad scope of SRE responsibility requires tight collaboration and alignment, which must occur at all levels: within the SRE team itself, within the larger SRE organization, and with the relevant software development teams.

Within a team

We often describe silos between functions or organizations, but silos can also exist within teams in the form of isolation of information, expertise, or skills. Common reasons for formation of knowledge silos are hoarding knowledge for job security, ignorance (not understanding why collaboration is beneficial), distrust (not believing that the potential partnership would actually be helpful), or just a lack of time or clear expectations. To overcome a tendency toward siloization, it is important to create a culture where people benefit from sharing information. Learning-focused organizations encourage a growth mindset, investing in continuous improvement rather than expecting people to know everything. At Google, we strive to actively reward collaboration through recognition and promotion. To achieve this, we encourage transparency and broad knowledge sharing through various strategies.

One mechanism to share updates and review reliability issues is a weekly *production meeting* (see Chapter 6) where SRE team members update each other on previous incidents and learnings as well as upcoming production changes, launches, and tests.

Another strategy is weekly *training sessions*, where different team members present on products, technologies, SRE fundamentals, personal initiatives, and so on. In addition to knowledge sharing, training sessions enable personal growth through teaching, improving on-caller preparation, and encouraging a culture of learning.

A third mechanism for breaking down knowledge silos and sharing expertise is participating in simulations like the *Wheel of Misfortune* (see Chapter 5).

We strive to avoid relying on the *heroics* of individuals who, when needed, make extraordinary efforts on their own to solve or prevent complex issues. While often satisfying and subject to praise in the moment, this behavior is bad in the long run and leads to several SRE antipatterns:

- It masks systemic problems, which never get fixed. The team isn't motivated to set a realistic SLO or to work on long-term systemic improvements.
- It cultivates a team culture that reinforces unrealistic expectations about the work expected from team members.
- It's unsustainable and leads to burnout.
- It leads to siloing.

Within the broader SRE organization

Junior SREs benefit from interacting with more experienced SREs across the organization to learn methodologies and best practices. They can also observe and replicate experienced SREs' personal and organizational growth strategies. Where possible, geographical colocation positively contributes to this learning.

Similar to team-level production meetings, Google has site-wide weekly *production meetings*, where teams share on-call challenges and learnings. This includes discussion of previously handled incidents, new project designs, and new automation beneficial to SREs (see the sidebar "Production Meetings"). In addition, SRE leadership regularly publishes a company-wide report titled "Google's Greatest Hits and Misses": a transparent and easily readable quarterly review of the most severe user- or revenue-impacting incidents, aiming to teach engineers how to prevent outages and build reliable systems.

Tech talks where subject matter experts in one team give a presentation or demo to the broader organization also foster collaboration, shared responsibility, and the exchange of knowledge. We'll dig into this topic more in Chapter 5.

Outside SRE

Google intentionally separates SRE from the development function, enabling autonomy—SRE teams don't report to dev leads (Chapter 6). The goal is to foster close collaboration between SRE and dev as equal partners.

A *user-focused* culture provides a common framework for SRE and development to maintain *appropriately reliable* systems, where appropriate reliability is defined by customer needs. Business owners have to define the "right" level of reliability for a given product,

and the organizational culture should enable both developers and operators to enforce it.

The SRE and development organizations at Google are intentionally structured to balance two critical priorities: the primary focus of development is to deliver features that contribute to the product's functionality, and the primary focus of SRE is to deliver reliability (itself a mission-critical feature). Both work toward the common goal of creating a valuable and stable product, and both roles must feel a sense of product ownership and align decisions and priorities to business goals. For example, a well-established product requiring five nines of reliability will have purposely slow rollouts with automatic rollbacks and extensive canarying. Conversely, with new products, customers are often more interested in seeing new features fast and may be more tolerant to outages, so the dev and SRE teams might decide on a lower SLO to shift the balance toward velocity and innovation. However, this doesn't mean that development can or should unilaterally ignore reliability or brush SRE considerations aside due to the fast-moving nature of the emerging product; instead, they should work together with clear agreement on the trade-offs being made and why.

SLOs and error budgets allow business, dev, and ops teams to track progress toward agreed-upon business goals. We all want to balance development velocity with reliability in a way that delivers maximum value to our users.

Collaboration between SREs and developers is further driven by dev participation in production meetings, collaboration on incident and postmortem reviews, and SRE-dev leadership *operational reviews*, where the teams jointly review production metrics and leading indicators.

A frequent point of contention is how early SRE should be involved in the design of a new product. Late involvement can lead to SRE reviews delaying launches if the risks raised about designs or operations don't meet reliability standards. As reliability experts, SREs must be able to consult on the reliability and scalability of future products. Also, as production owners, SREs must be aware of new developments ahead of time, to allocate human and compute resources needed for productionization and support.

Learning and continuous improvement

> *Never let a good crisis go to waste.*
>
> —Widely attributed to Winston Churchill

Innovation, such as adding new features, inherently involves change and risk, making some level of system failure inevitable. We see outages as valuable opportunities to learn and improve (Chapter 11), but this is possible only if we identify the procedural and/or systematic causes. *Postmortems* are a formalized process for learning from outages. They provide a broad-based mechanism to prevent and reduce recurring incidents, or at least reduce their likelihood and impact. We internally publish a written record of the incident, including mitigation actions, impact, root cause(s), and follow-up actions to prevent recurrence. Additionally, we share postmortems widely to help teams collectively learn from outages, translating lessons into concrete actions for improvement and enabling others to prevent similar incidents in their own systems.

Good postmortems are blameless, preventing conversations about who's at fault. Rather than seeking to attribute blame, the focus is on understanding and improvement.

The postmortem process is just one example of a mechanism to drive continuous improvement and underpin the high standards of excellence that SRE engenders. At its core, learning and continuous improvement is about leveraging SRE's data-driven and software engineering mindset to fundamentally improve the systems themselves, to make them more autonomous and help prevent future outages. This thread is woven throughout a number of chapters in this book, including Chapters 5, 11, 12, and 13.

Blamelessness and blame awareness

Blameless postmortems are one aspect of a blameless culture. Creating a safe environment and fostering a culture of learning requires understanding blame and acknowledging cognitive biases. Blame can be a way to discharge discomfort, as Brené Brown suggests. It's crucial to recognize cognitive biases such as the fundamental attribution error (where we attribute others' actions to internal traits and our own to external circumstances) and hindsight bias (the tendency to see past events as predictable).

Blamelessness shifts the focus of responsibility from *people* to *systems and processes* after an incident. Finger-pointing leads to risk aversion, fear, and a tendency to hide mistakes, which reduces situational awareness, increases the risk of problems accumulating, and limits the organization's ability to be proactive—stunting the ability to innovate and improve. A blameless culture instead focuses on identifying and improving the systems and processes that *allowed* the incident to occur. It assumes that individuals act in good faith and make decisions based on the best information available.

Blame awareness takes this a step further. It acknowledges that blame is a natural human reaction, but instead of ignoring it, it encourages a conscious and curious examination of

why an action that seems like a mistake made sense to the person who took it at the time. It asks, "Given the circumstances, the available information, and the pressures of the moment, why was this a reasonable thing to do?" This approach seeks to understand the "why" behind the "what," leading to a deeper understanding of systemic flaws.

This shift from blame to understanding, however, should not be treated as zero accountability. While postmortems do not focus on human errors committed during an outage, it is important that everyone is held to high engineering standards. We have expectations for everyone in Google engineering, and these are especially critical when working on systems that have a wide scope of impact to Google's users and its business. Postmortems should assess whether appropriate engineering standards were observed in both code and system operation, and prescribe appropriate follow-ups where deficiencies are discovered.

WHAT IF AN EMPLOYEE ACTS MALICIOUSLY?

A common concern that we hear from those aspiring to adopt SRE principles, particularly in finance and regulated industries, is: what if an employee undertakes a malicious action (e.g., stealing)? Isn't blame or holding people strictly accountable appropriate in that case?

The principles of *just culture* can distinguish between honest mistakes, at-risk behavior, reckless conduct, and plain criminal cases. The latter two are rare instances where disciplinary action or legal consequences are appropriate. However, when referring to day-to-day employee actions, we want to avoid blame to encourage appropriate risk taking and innovation. Even in cases that cross a line, it is important to do a postmortem and ask: what can we improve in our systems to make it impossible for someone to cross that line? Are we using the principle of least privilege? Do we have auditing enabled on those resources? Do sensitive operations have appropriate review and approval processes?

Psychological safety

According to Dr. Amy Edmondson, *psychological safety* is the belief that one will not be punished or humiliated for speaking up with ideas, questions, concerns, or mistakes.

A blameless culture is crucial for establishing this condition. With psychological safety, people feel comfortable escalating incidents early, reporting problems, asking questions to identify root causes, and sharing information. Without it, the entire incident response

lifecycle breaks down—mitigation might be delayed, resolution might take longer, and the critical learning phase will suffer (see Chapter 10).

Moreover, psychological safety is a critical enabler of a culture of continuous improvement. When team members feel safe, they are empowered to question the status quo and suggest better ways of working, far outside the context of outages and postmortems. This manifests in questions like: "Why do we have to do this toilsome task this way? Does the system architecture make sense? Is it okay that we haven't tested our backups? What if…?"

Managers and organizations play a key role in creating an environment where people feel safe and empowered to voice ideas. This includes promoting psychological safety by acknowledging one's own fallibility (e.g., saying "I didn't understand"), modeling curiosity by asking questions, leading by example, encouraging others, and not discouraging questions from others (e.g., through bad habits like feigned surprise). Managers are explicitly responsible for fostering psychological safety by framing work as a learning opportunity and not just an execution problem. People must feel confident that asking questions or requesting help will be rewarded, not punished.

Trust

> *Trust is a fragile thing—hard to earn, easy to lose.*
>
> —M. J. Arlidge

To freely speak up about mistakes, people need *trust* in their team, their manager, and the organization—trust that their input will be used for system improvement, not for blame or punishment.

The following paragraphs describe several strategies that can help leaders establish trust between people in their organizations.

One strategy is to create an organization where people choose to operate from the "assume good intentions, where possible" mindset. This enables people to safely assume that their colleagues are competent and trustworthy. Note that fostering a culture that assumes employees are competent and trustworthy doesn't mean you never address performance issues. However, it does mean that you evaluate performance objectively, without initial assumptions.

Multiple studies suggest that people need to experience at least five to seven positive exchanges to repair the trust damage that a single tense incident can cause.[1] At Google,

we try to maximize positive interactions to build the "assume good intentions" mindset, so when a later disagreement or a conflict occurs, people remember their colleagues' good sides, don't jump to negative assumptions, and are able to resolve the issue in a healthy and constructive way. For globally distributed SRE teams, targeted in-person meetings (in addition to virtual interactions) can be a crucial investment in this trust reserve (see Chapter 6).

Leaders at various levels enable trust by extending it to their reports, acknowledging their own mistakes, and following up with accountability. As mentioned previously, leaders should also model blamelessness, rewarding people for acknowledging and resolving issues. At the team level, managers can build trust and community through collaboration activities and exercises. Effective, work-focused approaches include workshops to define team-specific communication styles and norms, and collaborative exercises like practicing incident response scenarios together. These activities help team members understand how their colleagues approach problems and communicate under pressure, building rapport and mutual respect through shared professional experiences. Such approaches also improve psychological safety, performance, and retention.

Empowerment

It's important to empower SREs to speak up when reliability is threatened by a poor design or launch plan, even before metrics like error budget are impacted. SREs also need to regulate their operational workload. When system performance is not meeting expectations, they must advocate for spending more time on reliability, even if it means slowing down feature releases.

To prevent burnout from SRE teams getting buried in operational tasks, it's vital to set dedicated time for reliability work during the development cycle and not treat it as an afterthought. This work helps SREs advance their careers and deliver value to the business.

Empowerment goes further than this, however. SREs must be in a position to understand and fix reliability issues in a system. SRE is also a culture of initiative. SREs are not only allowed but also expected to drive continuous improvement.

At Google, one way this empowerment is manifested is through access to the code and the ability to fix things in the codebase. Any member of the team (regardless of role) may propose a code change or fix. Once that change is reviewed by the code's owner and approved, it is submitted.

One long-time Googler recalls their time as a newly hired SRE. They discovered an issue in the org-wide onboarding checklist on one of their first days. They brought up the issue with their tech lead, who said, "Let me show you where the checklist code resides. You can fix it yourself. This will be your first code submission."

Empowerment also manifests in the culture of learning from failures. SRE teams can request postmortems for any technical (or even organizational) issues, including near misses, and all the involved parties will engage in the investigation to uncover systemic causes. They can also assign follow-up action items to other teams and leaders to ensure the lessons learned lead to concrete improvements and prevent recurrence.

At Google, SRE teams are empowered to manage their own operational workloads as well. If a team is spending too much time on manual, repetitive tasks, the SRE manager can work with the corresponding dev team manager to push back on new feature releases and dedicate time to automation and other reliability improvements. In some extreme cases of service instability, they can also decide to hand the pager back to the development team.

The goal is to build an organization where people have a sense of ownership over reliability and are empowered by leadership to inquire, respectfully challenge others, and continuously improve the system and related processes. This requires a culture where people are motivated to break down silos, collaborate, speak up about mistakes without fear of punishment, and work on reliability, confident that this work will be recognized as having the same value as a shiny new product feature.

DORA's Findings on Culture and Performance

DevOps Research and Assessment (DORA) is a comprehensive research program dedicated to understanding the capabilities that drive success in technology-driven teams and organizations. Through annual global surveys of technical professionals, DORA investigates how technical practices, lean management, cultural norms, and leadership influence IT performance, software delivery, and overall organizational outcomes. A key aspect of DORA's findings, often framed through the lens of Ron Westrum's typology of organizational culture, highlights the profound impact of culture on performance. Westrum's model categorizes cultures into power-oriented (pathological), rule-oriented (bureaucratic), and performance-oriented (generative), demonstrating how information flow and processing within an organization are crucial predictors of safety and success. This systematic, evidence-based approach provides a foundation for identifying the most

effective practices for achieving desired results, particularly in relation to SRE practices and high-performing teams.

The Empirical Link: Culture and Performance

A consistent and crucial finding throughout DORA's research is the significant link between organizational culture[2] and various performance metrics. Specifically, a reliability-focused culture helps improve performance outcomes. DORA has empirically demonstrated that organizational culture predicts both software delivery and operational performance (collectively known as *SDO performance*) and overall organizational performance. A high-trust culture that encourages a climate of learning and collaboration has been identified as one of the clearest predictors of performance. Moreover, DORA's research correlates high performance with specific cultural aspects such as flexibility, knowledge sharing, and user-centrism.

Beyond performance metrics, a healthy culture contributes to better well-being outcomes, including increased job satisfaction, less burnout, and higher productivity. A healthy organizational culture can also help teams be more successful at implementing technical capabilities associated with improved outcomes.

In short, research demonstrates that culture is foundational to building technical capabilities, improving technical performance, reaching organizational performance goals, and helping employees succeed.

SRE Practices, Reliability, and Practitioner Well-Being

DORA's research has shown that teams that excel at modern operational practices, which include SRE, are 1.4 times more likely to report greater SDO performance and 1.8 times more likely to report better business outcomes.[3] A shared responsibility model of operations, where developers and operators are jointly empowered, also predicts better reliability outcomes.[4] The research indicates a relationship between SRE practices and practitioner well-being as well:

> *Beyond improving objective measures of performance, SRE improves technical practitioners' experience of work. Typically, individuals with a heavy load of operations tasks are prone to burnout, but SRE has a positive effect. We found that the more a team employs SRE practices, the less likely its members are to experience burnout. SRE might also help in optimizing resources: teams that meet their reliability targets through the application of SRE practices report that they spend more time writing code than teams that don't practice SRE.*[5]

Another interesting finding is that teams with quality documentation are 3.5 times more likely to implement SRE practices.[6]

Westrum's Typology of Organizational Culture

As noted in "The Case for Culture", organizational culture is the bedrock of an SRE practice. DORA's research utilizes a typology developed by sociologist Ron Westrum[7] to provide a framework for understanding organizational culture. Westrum's work examined organizational culture as a predictor of safety, particularly in high-risk domains like healthcare and aviation, highlighting the crucial role of the way organizations process information. DORA adapted this model to study culture in the context of technology-driven organizations.

Westrum's model identifies three primary organizational culture types based on how information flows and is processed (see Table 4-1).

Table 4-1. Characteristics of power-, rule-, and performance-oriented culture

Power-oriented (pathological)	Rule-oriented (bureaucratic)	Performance-oriented (generative)
Low cooperation	Modest cooperation	High cooperation
Messengers punished	Messengers neglected	Messengers trained
Responsibilities shirked	Narrow responsibilities	Risks are shared
Bridging discouraged	Bridging tolerated	Bridging encouraged
Failure leads to scapegoating	Failure leads to justice	Failure leads to inquiry
Novelty crushed	Novelty leads to problems	Novelty implemented
Leader's focus: personal needs	Leader's focus: departmental turn	Leader's focus: the mission

Let's look at some possible responses to an incident in each of the different organizational types. As shown in Table 4-1, Westrum's typology defines three organizational cultures—power-oriented (pathological), rule-oriented (bureaucratic), and performance-oriented (generative)—based on how information is processed and cooperation occurs. Suppose our fictitious organization has recently experienced an incident during which customers were unable to complete transactions and the business missed out on significant revenue. A recent change to how the application works was identified as a con-

tributing factor to the incident. Here's an example of how each type of organization might respond:

Power-oriented (pathological)

> The engineer who led the design of the new feature attempts to shift the blame away from the feature to another area of the application: "My design isn't flawed; it's the database performance that caused the incident!"

Rule-oriented (bureaucratic)

> The engineering manager calls for an audit of all handoffs and documentation to prove that the engineering team followed all prescribed processes and procedures leading up to the incident. The manager then suggests that someone on the database team should be reprimanded.

Performance-oriented (generative)

> A cross-functional team comes together to better understand the nature of the outage. The team looks for ways to improve the performance of the new feature as well as process improvements that will allow for faster feedback on potential performance bottlenecks.

DORA's research measures organizational culture by assessing attributes aligned with this typology. The findings consistently highlight a generative culture as being predictive of positive outcomes in software delivery and organizational performance.

Characteristics of a Generative Culture

DORA's research and the Westrum model detail the hallmarks of a generative organization that contribute to its high performance and positive outcomes. These characteristics map closely to the principles that underpin successful SRE practices, as outlined in Table 4-2.

Table 4-2. Comparison of Westrum's Generative and Reliability-Focused Cultures

Westrum's generative culture	Key elements of a reliability-focused culture
Good information flow (messengers are trained)	Empowerment and trust, a data-driven mindset
High cooperation and trust	Collaboration and shared responsibility
Bridging between teams	Collaboration and shared responsibility
Conscious inquiry (failure leads to inquiry)	Blamelessness, learning and continuous improvement
Novelty is implemented	Learning and continuous improvement
Risks are shared	Collaboration and shared responsibility
Leader's focus: the mission	Reliability as a core value and focus on the user

Let's explore each of the items in Table 4-2 in a bit more detail:

Good information flow (messengers are trained)

In a generative culture, information flows freely and is actively sought out. Bad news is not suppressed or punished; instead, messengers are seen as valuable for bringing problems to light so they can be addressed. DORA connects this directly to practices like holding blameless postmortems, where the focus is on understanding system and process failures, not blaming individuals. Removing blame reduces fear, enabling teams to surface and solve problems more effectively.

High cooperation and trust

Generative organizations are characterized by high levels of cooperation and trust across teams and functions. This aligns strongly with the SRE emphasis on collaboration between development, quality assurance, operations, and information security. A high-trust culture allows people to experiment freely. DORA's research has echoed findings from other studies, including Google's Project Aristotle, that psychological safety—feeling safe to take risks—is the most important factor for a high-performing team. A culture that values psychological safety, trust, and respect contributes to productivity by allowing employees to focus on solving problems rather than navigating organizational politics.

Bridging between teams

Generative cultures actively encourage bridging, breaking down silos between different parts of the organization. This can be fostered through creating cross-functional teams, colocating different roles, or including more teams throughout the software development lifecycle. Informal discussions and collaboration between teams can also foster better communication and creative problem-solving.

Conscious inquiry (failure leads to inquiry)

Rather than assigning blame when failures occur, generative organizations see them as opportunities for conscious inquiry. How an organization responds to a failure is crucial; when the response leads to questions about systemic issues and prevention, it strengthens the technical system, processes, and culture. Blameless postmortems are a key practice here, ensuring that learning and improvement, rather than punishment, are the outcomes of incidents. See Chapters 10 and 11 for more information.

Novelty is implemented

Generative cultures embrace and implement novel ideas, encouraging experimentation and a strong focus on continuous improvement. Giving employees the freedom to explore can lead to innovation and significant value for the organization. This applies not just to new products or features, but also to improvements in processes and collaboration.

Risks are shared

Responsibility and risks are shared across the organization in a generative culture. Quality, throughput, reliability, and security are seen as everyone's job. Practices such as developers sharing responsibility for maintaining code in production can improve collaboration and reduce risk. Automation also plays a role in risk reduction. See Chapter 8 for more information about how SREs share risks with other teams.

Leader's focus: the mission

In a generative culture, a leader's primary focus is a well-defined mission. SRE views SLOs not as arbitrary targets but as a reflection of the underlying mission to make users happy.

These characteristics align strongly with the ethos of SRE, which emphasizes shared ownership of reliability, learning from incidents through blameless postmortems, cross-functional collaboration, and continuous improvement. DORA's research validates that a culture exhibiting these traits is not just nice to have; it is a predictor of success in the

very areas SRE aims to improve: software delivery speed, stability, organizational performance, and even the well-being of the technical teams. This commitment to continuous learning and adaptation is at the heart of DORA's philosophy: "Get better at getting better."

How Learning Drives Culture

As we saw in the previous section, SDO performance and organizational performance are intrinsically linked to culture. DORA emphasizes that improving technology, processes, and capabilities alone is insufficient for success without a healthy culture.

Learning drives culture by catalyzing a chain reaction. *Learning drives confidence, confidence drives behavior, and behavior, repeated over time, drives culture*.

Shifting organizational culture can feel impossible, especially when we think about it in the abstract sense—culture as a state of mind or state of being. As prominent MIT researcher and industrial anthropologist John Shook put it, "The way to change culture is not to first change how people think, but instead to start by changing how people behave—what they do." Stated another way: it's easier to act your way to a new way of thinking than think your way to a new way of acting.

How people work, what they *do*, is the key to changing culture, which is arguably a more tractable problem to tackle. In turn, confidence, a measure of self-efficacy or perceived capability, is a well-researched key to unlocking behavior change.

Training, in the context of adult learning, is not about spraying people with a fire hose of information and asking them to drink from it. Training is about *building confidence* and helping to fight imposter syndrome, particularly for individuals new to a team or the practice of SRE. Learning effectively acts as a "confidence engine" that can shift your reliability-focused culture into high gear.

Benchmarking Culture Through Confidence

Measuring and evaluating culture is inherently difficult. However, one "low-toil" approach is to benchmark culture by assessing the self-reported confidence of individuals in their ability to perform tasks related to desired cultural traits and technical practices. This can be done through a structured four-step process:

1. Defining survey questions

2. Grounding the rating scale
3. Measuring confidence over time
4. Analyzing/comparing results

At Google, for example, the SRE EDU Orientation program uses this method to measure its effectiveness in moving the needle on confidence. Reliability culture questions are included in both SRE EDU Orientation and general developer onboarding surveys. These questions ask participants to rate their ability to perform specific tasks related to key SRE cultural attributes, such as:

- Explain the benefits of a blameless postmortem culture (blamelessness).
- Measure my service's alignment with business goals using SLOs and error budgets (data-driven mindset).
- Use monitoring and alerting to make rational decisions about my service (data-driven mindset).

The questions are all action-oriented and tied to learning objectives ("A student should be able to..."). We'll cover this topic in more depth in Chapter 5.

To ensure that everyone thinks about these questions in the same way and to reduce variability in responses, it's important to use a grounded rating scale rather than a simple numerical one. A grounded scale provides specific descriptions for each point. Our five-point rating scale is grounded on measures of confidence with the following descriptions:

- No idea where to start
- Could do it with someone's help
- Could read docs and figure it out myself
- Could do it mostly from memory
- Could teach others mostly from memory

To understand how training impacts confidence over time, these questions are surveyed at multiple points:

1. Pre-orientation
2. Post-orientation
3. One month post-orientation
4. Six months post-orientation

In addition, we compare the survey results on self-reported confidence between participants who received training focused on reliability principles (SRE EDU Orientation) and

those who went through general developer onboarding—what Google calls engEDU Google Tech Immersion (GTI). engEDU GTI is a foundational curriculum that prioritizes Google's developer workflow and does not explicitly address reliability-focused concepts in depth. By collaborating with the engEDU team to ask the same three reliability-focused confidence questions at the same intervals, we can gauge how effective our SRE EDU Orientation program is at perpetuating Google's reliability-focused culture.

Table 4-3 summarizes the results of comparing the fraction of survey respondents who self-reported the highest levels of confidence ("Could do it mostly from memory" and "Could teach others mostly from memory") across the SRE EDU and engEDU GTI onboarding programs. The survey results demonstrate that SRE EDU participants consistently report higher confidence in reliability-related areas at various points during their ramp-up, compared to their peers who went through general developer onboarding.

Table 4-3. Relative performance of SRE EDU versus developer onboarding: benchmarking reliability culture through confidence

Aspect of reliability culture	Post	1 month	6 months
Explain the benefits of a blameless postmortem culture.	+23 pp	+19 pp	+28 pp[a]
Measure my service's alignment with business goals.	+27 pp	+15 pp	+34 pp
Use monitoring and alerting to make rational decisions about my service.	+49 pp	+26 pp	+43 pp

[a] *pp* stands for percentage points.

This data suggests that emphasizing reliability concepts as we do in SRE EDU Orientation is effective at building confidence on reliability-focused behaviors, which in turn reinforces our reliability-focused culture.

Tailoring Training to Different Audiences

The previous section shared a research-backed social science approach to tackling culture and demonstrated how training programs can perpetuate a positive culture by focusing on building confidence around a desired set of behaviors.

Training programs can also help drive a significant culture shift using the same *learning → confidence → behavior → culture* ladder. Understanding the current state of your

organization's culture is key when designing a training program.[8] Tailoring the content and approach based on this organizational context and the characteristics of the individuals being trained is important. Key dimensions to consider include:

1. The organization's maturity in adopting SRE principles
2. The individuals' prior experience with SRE practices
3. Team members' receptiveness to change

For organizations early in their SRE journey (low organizational maturity), addressing fundamental skill gaps is critical. This might involve training on how to use defect tracking systems, project planning models like Agile, version control systems, and shared tool repositories, as well as training on how to write design documents.

Understanding the existing team mix is also essential. Individuals can be categorized based on their SRE experience and receptiveness to new ways of working. For example:

- *Change curious* team members are often individuals who are new to the company and have less or no experience with SRE practices. They frequently ask questions like "Why do we do things this way?" and "Why is this process different from what I learned in orientation?" This curiosity can be helpful for driving behavior change across the organization that in turn drives culture change. Ensure that your training program matches the aspirational behavior you are striving toward, even if it hasn't been universally adopted yet.
- *Change resisters* are employees who are skeptical about the organization's desire to adopt SRE ways of working. To help them overcome their skepticism, these individuals typically want to know "What's in it for me?"
- *Change champions* are often individuals who are new to your company but have a high level of SRE experience from elsewhere. Give SRE change champions a platform to share their success stories (e.g., how SRE dramatically reduced instances of being paged in the middle of the night, how eliminating valueless toil gave them more free time to work on more interesting problems, or how they were able to improve the reliability of their services in a way that the business cheered). Storytelling can be a very effective vehicle for driving change and convincing the change resisters that this new SRE way of working is worth buying into.

Tailoring the format and content to these distinct needs helps ensure the training resonates with each group and maximizes your chances of successfully seeding and growing an effective reliability-focused culture.

Practical Considerations

If you are already working in an environment with a healthy, reliability-focused culture with many or all of the features described previously, you can probably stop reading here and skip ahead to the next chapter. However, if you are actively trying to bring SRE to your organization and bootstrap a reliability-focused culture by targeting positive behaviors, the sections that follow provide some practical advice on how to approach this.

The journey toward robust SRE practices and a reliability-focused culture doesn't always begin with a blank slate or an organization-wide, top-down mandate. More often, it starts with pragmatic, focused efforts within specific contexts. For many organizations, especially those outside the hyperscale world where SRE originated, the full breadth of SRE theory can feel overwhelming. The key is to distill its core principles into actionable, standalone strategies that deliver tangible value quickly, thereby building momentum and fostering cultural change organically. As we discussed in the previous sections, small changes in behavior, repeated over time, can catalyze significant changes in organizational culture.

Two primary pathways have proven effective for this incremental adoption: evolving reliability within existing "brownfield" systems and instilling it from inception in "greenfield" projects. Both approaches, when executed thoughtfully, can serve as powerful levers for introducing SRE concepts such as SLOs, error budgets, and a learning-oriented culture, even before a formal SRE function exists.

The Brownfield Evolution: Cultivating Reliability in Existing Systems

Most organizations have a portfolio of existing services, many of which are critical to the business and have been developed in-house over time. These "brownfield" systems, with their established operational patterns and, often, accumulated technical debt, might seem like challenging candidates around which to build a reliability-focused culture. However, they offer a rich environment for demonstrating the immediate benefits of SRE practices precisely because the impact of improved reliability can be so keenly felt. This helps build momentum. The core idea is to select a suitable existing service and use it as a pilot for SRE principles, working backward from an ideal state that is "reliable enough" given business objectives for current performance and then charting a plan of action to bridge the gap. In completing the steps outlined here, you are also seeding critical cultural elements, which should make additional change easier to achieve.

Educate your stakeholders

Before beginning to think about implementation, it's necessary to establish a foundational understanding of key concepts across developers, operators, and business representatives. A joint training, teaching the basics of reliability engineering to relevant stakeholders from all parts of an organization—everyone who is involved in the lifecycle of a service—has proven to be an effective initial step. This type of training also demonstrates learning and continuous improvement.

Select your starting point

Choosing the right service to begin your incremental journey to SRE is a crucial step. Not all services are created equal for this purpose. Ideal candidates typically exhibit one or more of the following characteristics:

Business criticality or visible pain

Pick a service whose reliability (or lack thereof) matters.

A reasonably engaged team

While you don't need full SRE converts, having development and operations teams (or a DevOps team) willing to experiment and try out new behaviors is beneficial.

Internally developed with code access

The ability to instrument and make code changes to the service is helpful and is one way to model the cultural tenet of empowerment.

In situations where an organization lacks the ability to instrument code—for example, adding a monitoring or logging library—developing this skill is helpful for building an initial reliability-focused culture because it lays the foundation for the service performance data necessary for informed decision making. This can be achieved through monitoring workshops and by implementing the appropriate tooling to collect application metrics (see Chapter 9).

Assess maturity and discover baseline reliability

Before diving into specific metrics, it is valuable to assess the service's broader operational context using a maturity matrix. As detailed in Chapter 7, SRE engagements do not always start with SLOs; sometimes, more prominent operational challenges—such as

excessive toil or unmanaged risks—must be addressed first. A maturity matrix allows you to evaluate where the organization stands across dimensions such as operational processes, risk management, and velocity.

Identifying whether a team is in a reactive phase (focused on fixing outages) or a proactive phase (managing dependencies and risks) provides the necessary grounding for a clear team charter and roadmap. These documents ensure that the SRE engagement solves the unique challenges relevant to that specific stage of the team's lifecycle, rather than applying a one-size-fits-all solution.

Once the maturity level is understood and a promising experimental service is chosen, the next imperative is to understand the service's specific performance and reliability state. Assumptions and anecdotal evidence are insufficient; you need data. It is here that defining and measuring SLIs directly tied to critical user journeys (CUJs) comes into play, which in turn reinforces the organization's focus on the user, which underpins the culture you are building.

This involves adding code or configuring tools to emit these metrics if it isn't happening already. Modern application performance monitoring (APM) tools, logging platforms, and time-series databases are invaluable here. For an internally developed service, developers can add custom metrics specific to CUJs or business transactions. For example, for an ecommerce checkout service, an SLI might be the success rate of "order placement" attempts.

WARNING

Defining the truly critical user journeys can be difficult, and adding instrumentation that directly measures a CUJ can be hard or impractical. You may have to rely on proxies that minimize the gap between what you can measure and what you wish you could measure.

Have the crucial conversation

This conversation should answer the fundamental question: is our current reliability good enough?

With baseline SLI data accumulating (ideally over a few business cycles to capture typical fluctuations), the next step is to bridge the technical and business domains. This involves presenting the current reliability picture, in SRE-focused terms, to business stakeholders, product owners, and key users of the service. This approach continues to build

the organization's muscle in focusing on the user and gives teams further practice in the behaviors that strengthen collaboration and shared responsibility between all parties.

This isn't about presenting a wall of dashboards. It's about a focused conversation: "What is the actual business impact when the service is slower than X, or unavailable for Y minutes, or Z% of transactions fail?"

Why does this matter? This conversation is foundational: it connects technical performance directly to business value and user satisfaction. It's also the point where the need for, and the targets for, SLOs emerge naturally. Often, there's a disconnect between what engineering *thinks* is acceptable and doable (a tendency toward underestimation), what business stakeholders *think* is necessary (a tendency toward overestimation), and what customers *actually* need or expect.

While securing leadership buy-in is vital, involving individual engineering and product team members in these crucial reliability conversations empowers them to genuinely value and understand how to prioritize reliability in practice, recognizing it as a paramount system feature that necessitates concrete action and investment of time and energy to achieve. Conducting these conversations helps the organization move away from a deploy-at-any-cost mentality toward a balance between velocity and stability, infusing reliability into the organization's culture as a core value.

Define initial SLOs

The insights from the business conversation directly feed into defining your first SLOs (see Chapter 8) for the brownfield service.

These SLOs represent a shared commitment to the reliability of the service, which should be implicitly or explicitly aligned with user expectations. They are also tied to correlation with other business metrics (e.g., user churn, revenue impact), reinforcing the organization's focus on the user.

Some characteristics of good initial SLOs for a brownfield service are:

Simple

Focus on two or three critical SLIs that emerged as important from the business discussion. Don't try to boil the ocean. Once the simple SLOs prove their value, you can iterate from there.

Realistic

Base them on current performance and what the business deems "good enough" or a reasonable stretch goal. If your current availability is 99.5%, setting an initial SLO of 99.999% is likely setting yourself up for failure. Learning and continuous improvement is key and requires the entire organization to work toward the new target SLO.

Understood and agreed upon

Both engineering and business stakeholders should understand what the SLO means and agree it's a worthwhile target. Getting to this agreement again reinforces the collaboration and shared responsibility that underpin a healthy reliability-focused culture.

Introduce blameless postmortems

With an SLO in place, any significant deviation (an SLO breach or a notable incident affecting the service) becomes a prime opportunity to introduce another core element of a reliability-focused culture: blamelessness, which can be channeled explicitly through the action of writing a blameless postmortem.

After an incident is resolved and the team enters the learning phase (see Chapter 10), the assessment process must model and reinforce psychological safety. The operational goal is to learn and improve the system (see Chapter 11), not to identify individuals to blame. Assume everyone acted with good intentions based on the information they had at the time.

Spread the word and iterate

As you implement these steps for the brownfield service—measuring SLIs, defining SLOs with business input, and learning from incidents blamelessly—you are not just improving that one service. You are building momentum for your reliability-inspired culture:

Showcase wins

Share the progress. "By making our new latency SLO for service X a priority, we identified and fixed a bottleneck, improving user experience by Y% and reducing complaints by Z%."

Iterate

Revisit SLOs periodically. Are they still the right ones? Are the targets appropriate? As the service evolves, so should its metrics (SLIs) and reliability goals.

Expanding the culture

The success you demonstrate and the structured, data-driven operational approach taken with this pilot service can inspire other teams to ask, "How can we do that for our service?"

The Greenfield Design: Engineering Reliability from Day Zero

The second powerful approach to incremental SRE adoption is in "greenfield" projects—designing new systems with reliability as a core nonfunctional requirement from the beginning. This offers a different set of advantages, including the ability to budget for and bake in best practices without battling existing architectural constraints or entrenched operational habits. By building the muscle memory around key behaviors from day zero, you can ensure your reliability-focused culture will grow and scale alongside your greenfield service. This section outlines the steps to follow with this approach.

Use SLOs as a design input

In a greenfield scenario, the conversation about SLOs happens *before* any significant code is written or infrastructure is provisioned. This means you can set the stage for embedded reliability as a core value and a strong focus on the user right from the start.

Just as with the brownfield approach, product owners, business stakeholders, and potential users are involved early, practicing collaboration and shared responsibility. The discussion with these stakeholders, however, is forward-looking, asking questions like:

- "For this new service to be successful and make our customers happy, what level of availability will it need?"
- "What are the critical user journeys, and how fast do they need to be?"
- "What level of errors can users tolerate before they lose trust or abandon the service?"

Additionally, SLOs drive architecture. These co-defined SLOs become fundamental inputs into the system's design. If a service requires 99.99% availability, this might lead you away from certain single-homed architectures and toward designs incorporating redundancy, automated failover, and advanced incident response. If low latency for a particular API endpoint is critical, this informs choices about caching strategies, database technologies, and data locality. Using SLOs as design inputs brings a data-driven mindset to system design viewed through a reliability-focused lens.

Engineer the system to meet the SLOs

With SLOs defined, the engineering team has clear reliability targets that it is empowered to design and build against. This influences several aspects that ultimately let the team practice behaviors that embed reliability as a core value in the product and organization:

Technology choices

Selections of databases, messaging queues, load balancers, and even programming languages or frameworks might be influenced by their known reliability characteristics and support for observability.

Architectural patterns

Teams can design for fault isolation (so one component failure doesn't cascade), redundancy at critical points, and graceful degradation (so noncritical features can be disabled under load to protect core functionality). In such decoupled systems, where retries between components become more common for reliability, idempotent operations—ensuring that requests processed multiple times have the same effect as a single execution—become conscious design choices.

Instrumentation as a feature

SLIs are not an afterthought. Monitoring and observability hooks are built into the service as it's developed. Dashboards for tracking SLO compliance can be created alongside the features themselves.

Testing for reliability

This includes not just functional testing but also performance testing, load testing to validate that the system behaves as expected, and potentially chaos engineering experiments (on a small scale, initially) to uncover unexpected behavior under stress and failure conditions.

Establish operational readiness from the start

A greenfield project also provides the opportunity to establish sound operational practices from day one that strengthen the reliability-focused culture you are building. These include:

Automated deployments

Implement robust CI/CD pipelines that include automated testing and safe roll-back mechanisms.

Clear runbooks and playbooks

Document how to operate the service, diagnose common issues, and respond to alerts. This documentation can be simple at first.

Proactive capacity planning

Make initial estimates for resource needs based on expected load and SLOs.

BROWNFIELD VERSUS GREENFIELD APPROACHES

The greenfield approach avoids the "technical archaeology" often required in brownfield projects. There's less unlearning of old habits and more opportunity to establish good SRE-aligned practices from scratch. However, it can be challenging to accurately predict usage patterns and failure modes for a brand-new service.

By launching a new service that is well-instrumented, has clear reliability targets co-defined with the business, and is designed to meet them, you create a powerful internal showcase of "SRE thinking" in action, which in turn reinforces a robust reliability-focused culture.

Error Budgets: SLO-Driven Operations

While establishing SLOs for a brownfield service or designing them into a greenfield one is an excellent first step, the concept of an error budget is what makes an SLO actionable and transformative for day-to-day decision making. Error budgets are meant to mediate the collaboration and shared responsibility at the core of a reliability-focused culture by balancing the desire for rapid innovation with the need for stable, reliable services.

Using the error budget as a decision-making tool can help guide release velocity and prioritization. A healthy budget allows for confident deployments, while a depleted budget signals a need to slow down or halt new feature development to focus on reliability work. Establishing an agreed-upon error budget policy formalizes this approach.

Implementing error budgets necessitates a cultural shift in how teams allocate time and effort, and as a result it takes time and effort. Regular review of error budget consumption ensures alignment and drives proactive improvement efforts, even when the budget

is healthy. This helps establish or reinforce reliability as a core value by embedding reliability work directly into team priorities and daily operations.

WARNING

While error budgets can build momentum for a reliability-focused culture, they aren't a panacea for all things reliability-related. For a more detailed discussion of SLOs and the mechanics and potential pitfalls of error budgets, see Chapter 8.

Revised Processes: Moving Beyond Cadence and Promises

The introduction of SLOs and error budgets, through either brownfield evolution or greenfield design, naturally leads to—and in fact, necessitates—a revision of existing operational and development processes. The old ways of working often clash with SRE's data-driven mindset and emphasis on reliability as a core value. By *changing the way people work* in the ways described here, you can effectively set up a flywheel for a positive shift in organizational culture.

From cadenced deployments to SLO-gated deployments

Many organizations operate on fixed-cadence deployments (e.g., "we deploy every two weeks on a Tuesday"). While predictable, this model often leads to changes being pushed out regardless of the current stability of the system or the risk appetite defined by an error budget. It also fails to take into account whether the code is ready for production. Business-mandated deployments, driven by feature promises or marketing commitments, can override stability concerns, leading to a "deploy at all costs" mentality. On the flip side, they may slow down development teams capable of a higher release frequency. This approach discourages many aspects of a reliability-focused culture, including collaboration and shared responsibility, empowerment, a data-driven mindset, and learning and continuous improvement.

With SLOs and error budgets, deployment decisions become more nuanced and reinforce a reliability-focused culture. The central questions before proceeding with a deployment are:

- Is the error budget healthy? If yes, proceed with the deployment, assuming it has passed all other quality gates (data-driven mindset).
- Is the error budget low or exhausted? If so, the default response should be to delay the deployment and focus on restoring the budget (data-driven mindset, collaboration, and shared responsibility).

This doesn't mean no features ever get deployed if reliability is shaky, but it forces an explicit, data-informed conversation about risk versus reward. Sometimes a critical bug fix *must* go out, and that's an accepted use of the (remaining) budget, or we make a conscious decision to go slightly over the budget, understanding the consequences.

Feature promises versus reliability guarantees

Product roadmaps are often built on a series of feature promises with deadlines. Engineering teams are pressured to meet these deadlines, sometimes by cutting corners on testing, nonfunctional requirements, or observability.

SLOs are, in effect, reliability guarantees to the users and the business. The error budget provides the mechanism to uphold these guarantees. Product and engineering must work collaboratively so that:

- Product understands that pushing for too many features too quickly can burn the error budget and lead to a (necessary) slowdown to maintain agreed-upon reliability.
- Engineering commits to the SLOs and uses the error budget to communicate when the system needs a "breather" to focus on stability.

This model also fosters a more dynamic relationship with reliability, as a healthy error budget can empower SREs to embrace experimentation and riskier launches. It helps shift the perception of ops teams from being solely gatekeepers to active enablers of innovation and velocity.

Once again, we see how SLOs and error budgets incentivize the behaviors that lead to a robust reliability-focused culture.

Shifting team focus and effort

This new model requires a shift in how engineering teams allocate their capacity, which in turn reinforces the collaboration and shared responsibility tenet of a reliability-focused culture. Key considerations are:

Protected time for reliability

When error budgets are low, teams need dedicated time to diagnose issues, implement fixes, improve monitoring, or pay down technical debt that's contributing to instability. This time must be protected from the constant pressure of new feature development. Some organizations adopt guidelines to ensure a significant portion of engineering capacity is reserved (say, 50%). However, the key is that

this reserved capacity is flexible. When error budgets are healthy, this time is invaluable for SRE teams to proactively run projects, enhance automation, and contribute to shipping new features with a reliability mindset. When error budgets are strained, this same protected time is prioritized for restoring stability. Even dedicating a smaller explicit percentage of capacity for such focused SRE work initially (e.g., 20%) can help establish this crucial balance.

A proactive versus reactive approach

The goal is to move from a purely reactive mode (fixing things only when they break catastrophically) to a more proactive stance, where potential issues are identified and addressed *before* they cause major SLO breaches. Error budget trends can highlight these leading indicators.

By embarking on either the brownfield evolution or the greenfield design path, and by diligently implementing SLOs and respecting error budgets, organizations can make significant strides in improving service reliability.

These tactical approaches are not just about adopting specific SRE tools or techniques; they are, more importantly, about initiating a profound cultural shift toward data-driven operations, shared responsibility for reliability, and a continuous learning mindset. While the journey is incremental, the cumulative impact on service quality, operational efficiency, and, ultimately, customer satisfaction can be transformative.

Common SRE Implementation Challenges

In this section, we'll talk about some common SRE implementation challenges and how you might address them as you lay your cultural foundation.

Getting Executive Support

Planting the seeds and growing a reliability-focused culture may be viewed as "fluffy" (i.e., unimportant or a waste of time) rather than as an essential tool for reaping the tangible benefits of SRE that we discussed in Chapter 2. Leaders need to be convinced to walk the talk to pave the way for a successful SRE implementation. Simply renaming an existing team to "SRE" without incentivizing behaviors in line with SRE practices and principles can poison the organization against the entire concept of SRE in the future, so it's extremely important to have buy-in from your leaders.

Based on the anecdotal experiences reported by a variety of customers using Google Cloud, without executives driving the culture from the top, it's much more difficult to build up SRE practices. A culture mismatch between leadership and the teams trying to establish SRE practices can lead to tension and potentially a shift away from SRE. SREs must also be empowered to make decisions. Errors and breakages are inevitable, but if your leaders are still trying to find someone to blame when incidents happen, it could very well inhibit progress. Disempowerment, distrust, and blame are all reliability culture antipatterns, after all.

So what can you do to convince your execs to invest in and support organizational cultural improvements? As noted in "DORA's Findings on Culture and Performance", DORA provides a research-backed and industry-recognized toolkit to help you convince your stakeholders of the value of SRE. Specifically, DORA illustrates how a healthy culture:

- Improves performance outcomes
- Can predict SDO performance
- Results in more job satisfaction, less burnout, and higher productivity

DORA research has also demonstrated that a healthy organizational culture can help teams be more successful at implementing technical capabilities associated with improved business outcomes.

Managing Tooling

As we saw earlier in this chapter, actions and behavior drive culture. Tooling can heavily influence the practice of Site Reliability Engineering and, more importantly, a reliability-focused culture. Tooling can impede SRE implementations in a variety of ways. Potential traps include:

- Using tools incorrectly (e.g., monitoring the wrong components)
- Using the wrong tools (so you're not able to collect the right signals)
- Being unable to get funding (so you can't get the right tools for the job)
- Getting blocked by resistance to change

Every organization is different, but SRE principles remain the same. It's not necessarily about getting a set of specific tools in place; instead, it's about using the best tools for your scenario.

Changing tools is not a simple process, but there are benefits to using the right tools based on your operating model and goals. Let's explore why you might consider switching tools as part of your plan for adopting SRE.

New tools can drastically improve efficiency and productivity, leading to significant time savings. This frees up valuable time for more important engineer-focused work driven by an empowered team.

A key aspect of cultivating an engineering-driven and reliability-focused culture is the ability to automatically detect and prevent issues as early as possible while minimizing human toil. Selecting the right tooling is critical to reinforcing the behaviors that support that culture. For instance, adopting static analysis tools in CI/CD pipelines can help identify common errors and security vulnerabilities, preventing developers from introducing defects into the system.

Similarly, a data-driven mindset necessitates having readily available information about SLO compliance at any given point in time. This is another place where good tooling can reinforce a reliability-focused culture. Tools that provide data and structure can help teams make better, data-driven decisions about their SLOs and error budgets.

A reliability-focused culture prioritizes the user and thus values rapid recovery and customer satisfaction. Tools that assist with mitigation during an incident are essential, as they are designed to speed up recovery (leading to happier customers). Tools that enable easier system recovery keep reliability at the forefront of engineers' minds.

Once you find the right tools, standardizing their use across the organization can facilitate better communication and shared understanding between teams, which typically leads to enhanced collaboration and knowledge sharing. This reinforces learning and continuous improvement as part of the organization's culture. Using consistent tooling across SRE and product development teams is important to avoid divergence and maximize the benefits of tool improvement. It also enables more repeatable processes, noticeably boosting efficiency and often leading to shared practices and policies that support rapid growth and scale.

Standardizing tools can also help reduce costs and toil. In cloud environments, scaling options make it easy to throttle resource consumption when demand is low and scale up when it's required. Even organizations that do not fully standardize can still reduce costs by removing tools with overlapping capabilities. Once tools and processes are standardized, or at least aligned, common issues can be resolved more quickly or automated

away, allowing for more frequent deployments and shifting the focus to more important engineering work.

Although changing tools may feel like a huge undertaking, making the switch as part of the path to SRE adoption can contribute to building a more successful and functional team. Without the ability to track the right signals from your systems, perform fast rollouts and rollbacks, or implement appropriate security controls, it can be difficult to establish the foundations of a reliability-focused culture and meet your reliability goals. The transition may be tough, but the benefits typically outweigh the costs. With standardized tools in place, you can expect to reduce costs and increase velocity. Your teams will be more self-sufficient, feel more empowered, and perhaps be more self-regulating. This, in turn, should lead to more achievable reliability targets and happier users.

Managing Expectations

One last challenge to SRE implementation is managing expectations about what SRE can deliver for an organization and how much effort it will take.

Expecting perfection

> *100% is the wrong reliability target for basically everything.*
>
> —Ben Treynor Sloss

SREs in a well-established practice and culture understand that perfection is impossible, or at least cost-prohibitive, to achieve over any appreciable period of time. For organizations just getting started with SRE, leadership may believe the team is striving for perfection and anything less is a failure. This leads to a situation where the team can't avoid overpromising and underdelivering. Instead, it's important to educate the organization up front about the fact that SRE is about enabling maximum feature velocity while meeting the reliability needs of users.

Setting arbitrary reliability targets

To achieve the focus on the user that is so foundational to an SRE culture, customer feedback loops are absolutely necessary. Just because a product manager thinks the app should provide 99.99% uptime doesn't mean your users expect that. User feedback also drives a focus on realistic SLOs. It would be very difficult and expensive to operate every system at four nines of reliability (less than 5 minutes of downtime in a 30-day period). Your users determine your reliability needs, no one else.

Thinking there is an "easy button"

Some organizations assume SRE is easy. The truth is, it's a very complex undertaking. Often, operations improve somewhat after adopting SRE, but then an incident occurs. People scramble, and an outage may last longer than it should while the organization figures out how to operate in this new paradigm. However, teams learn by doing, and with each successive incident they make steady improvements that accelerate with time. How the organization reacts in these early days can make or break the SRE practice. As engineers practice key SRE behaviors (a form of on-the-job learning), their confidence grows, which in turn helps the reliability-focused culture to "stick" so that the organization can fully reap the benefits of operating in this new way.

Believing it's one and done

Once established, a high-performing SRE practice and culture must be nurtured. You need to flex the learning and continuous improvement muscles that are at the core of a reliability-focused culture. Make sure you are reviewing your SLOs at least once a year and using the feedback loops you have in place. SLOs may change over time, depending on how your system is being used. Good ideas can come from anywhere—from a brand-new employee to a veteran engineer with decades of experience—so establish forums to listen, collect ideas, and experiment. Although not a requirement, it's a great idea to rotate engineers in and out of different areas to provide different perspectives and the ability to learn new skills. Know the limits of your tools so you are aware of any blind spots in your systems, and use this knowledge to effect change. If possible, attend public conferences to get a broader sense of where the industry is going. Nothing is set in stone—it can't be if you want to be functional and successful. Site Reliability Engineering must be adaptable.

Conclusion

Organizational culture is foundational to a successful SRE practice. While SRE is an evolving discipline and its implementation naturally varies across organizations, the underlying principles of a reliability-focused culture remain constant. Whether your organization already has a dedicated SRE team, is starting to build one, or is just curious about SRE practices, we encourage you to focus on establishing and nurturing these cultural principles. Continuous improvement and empowerment to drive change are core to a reliability-focused culture, so set up feedback loops and mechanisms for teams to experiment to ensure that your culture continues to grow and evolve. If something isn't serving your current practice, shift in a direction that serves you better. Remember, learning

drives confidence, confidence drives behavior, and behavior repeated over time ensures that your culture will continue to thrive.

1 John M. Gottman and Robert Levenson, "Marital Processes Predictive of Later Dissolution: Behavior, Physiology, and Health," *Journal of Personality and Social Psychology* 63, no. 2 (1992): 221–33; Marcial Losada and Emily Heaphy, "The Role of Positivity and Connectivity in the Performance of Business Teams: A Nonlinear Dynamics Model," *American Behavioral Scientist* 47, no. 6 (2004): 740–65; Christian V. Sabey, Cade Charlton, and Shawn R. Charlton, "The *Magic* Positive-to-Negative Interaction Ratio: Benefits, Applications, Cautions, and Recommendations," *Journal of Emotional and Behavioral Disorders* 27, no. 3 (2018): 154–64.

2 For more on organizational culture, see Nicole Forsgren, Jez Humble, and Gene Kim, *Accelerate: The Science of Lean Software and DevOps* (IT Revolution, 2018), 29–40

3 Dustin Smith et al., *Accelerate State of DevOps 2021* (DORA, 2021), 19.

4 Ibid., 20.

5 Ibid.

6 Ibid., 21.

7 Ron Westrum, "A Typology of Organisational Cultures", *BMJ Quality and Safety* 13, no. 2 (2004): ii22–ii27.

8 Jennifer Petoff et al., *Training Site Reliability Engineers: What Your Organization Needs to Create a Learning Program* (O'Reilly, 2020).

Previous chapter
‹ II. Sociotechnical Foundations

Next chapter
5. SRE Hiring, Training, and Career Management ›